

Collision Resolution Techniques (Open Addressing & Chaining) -- Data Structures

Definition: A Collision occurs in hashing when a hash function maps two distinct keys \$k_1\$

to the exact same table index ($h(k_1) = h(k_2)$). Collision resolution techniques resolve this conflict.

1. Detailed Technical Explanation

COLLISION RESOLUTION STRATEGIES

1. OPEN ADDRESSING (Closed Hashing)

 - Linear Probing
 - Quadratic Probing
 - Double Hashing
2. SEPARATE CHAINING (Open Hashing)

 - Array of Linked Lists

2. Technique 1: Open Addressing

All keys are stored directly inside the hash table array. If slot $h(k)$ is occupied, systematic probing finds the next vacant slot.

1. Linear Probing

Probes slots sequentially with an offset of 1:

* **Drawback:** Suffers from Primary Clustering (long contiguous blocks of occupied slots build up, degrading search time).

2. Quadratic Probing

Probes slots using a quadratic polynomial:

* **Advantage:** Eliminates primary clustering; but can cause Secondary Clustering (keys with same initial hash follow identical probe sequences).

3. Double Hashing (Best Open Addressing Technique)

Uses two independent hash functions $h_1(k)$ and $h_2(k)$:

* **Rule:** $h_2(k)$ must never evaluate to 0 and must be relatively prime to m .

3. Technique 2: Separate Chaining

Each slot in the hash table points to the head of a Linked List storing all colliding keys mapped to that index.

Hash Table Array

Index 0: NULL

Index 1: [12] -> [23] -> [34] -> NULL (Collisions at index 1 chained)

Index 2: [13] -> NULL

Index 3: NULL

Separate Chaining vs Open Addressing Comparison:

Feature	Separate Chaining	Open Addressing
Storage Structure	Array of Linked Lists.	Single Array (Keys stored in slots).
Table Capacity	Can store $> m$ elements ($\alpha > 1$)	Limited to table size m ($\alpha \leq 1$)

Deletion	Simple node removal from linked list.	Complex (requires `DELETED` dummy mark
Cache Performance	Poor cache locality due to linked list	**Excellent cache locality** (contiguo

4. Quick Recall Flow

Collision ($h(k1) == h(k2)$) -> Open Addressing: Linear (i), Quadratic (i^2), Double Hashing ($i * h2$) | Separate Chaining: Linked List buckets